KDSH Track A: Technical Report

1. Overall Approach

We built a **Canonical Backstory** system which pre-generates authoritative character backstories from the novels, then compares input backstories against these canonical versions using an LLM(gemini).

Note on evaluation: Due to API rate limits, our evaluations used 10 rows (5 consistent, 5 inconsistent) that too from one book only.

How It Works

1. **Index novels**: We chunk novels into 1000-word segments with 200-word overlap, embed each chunk with Gemini's embedding model, and store them in PostgreSQL with the pgvector extension.

2. **Generate canonical backstories**: For each character in the dataset, we find all chunks that mention them by name, send these to an LLM, and ask it to summarize what the novel explicitly tells us about that character's backstory.

3. **Classify**: We pass both the input backstory and the golden story generated by LLM with a prompt to gemini and then ask it to compare them and classify it as consistent or inconsistent.

Why This Approach

We started with a standard RAG approach and achieved 50-70% accuracy. The problem was that retrieved chunks were topically similar but not always relevant to the specific claims in the backstory. The LLM would find spurious "contradictions" in passages that weren't actually relevant.

We felt the canonical approach worked better because:

- The LLM compares two backstories directly, not a backstory against raw text
- Comparison is more focused: "Does X contradict Y?" rather than "Does X contradict anything in these 5 chunks?"

2. Experiments and Findings

We ran multiple experiments to understand what affects accuracy. Here's what we tried:

Experiment 1: Baseline RAG

Setup: Chunk novels → Embed → Retrieve top-20 chunks → LLM classifies
Result: 50% accuracy
Finding: Random chance. The LLM was most probably overwhelmed by irrelevant chunks and couldn't distinguish signal from noise.

Experiment 2: Cross-Encoder Reranking

Setup: Retrieve top-20 chunks → Rerank with cross-encoder → Keep top-5 → LLM classifies
Result: 70% accuracy
Finding: This was our biggest single improvement in the RAG approach. The cross-encoder (ms-marco-MiniLM-L-6-v2) scores each (backstory, chunk) pair jointly, which captures relevance much better than embedding similarity alone.

Experiment 3: Prompt Engineering

We tried several prompting strategies:

Strategy	**Accuracy**
Base prompt	50%
Chain-of-thought	60%
Few-shot examples	50%
Conservative	62.5%
Claim extraction	40%

Finding: Conservative prompting helped. We explicitly told the LLM that absence of evidence is not evidence of contradiction, and that "seems unlikely" is not the same as "contradicts."

Experiment 4: NLI Models

Setup: Use DeBERTa-based NLI model to detect contradictions between backstory and chunks
Result: 26.5% accuracy
Finding: The NLI model was way too aggressive. It flagged almost everything as a contradiction.

Experiment 5: Hybrid NLI + LLM

Setup: Use NLI as a first pass, then LLM for uncertain cases
Result: 45% accuracy

Finding: The NLI was over-flagging, dragging down overall accuracy. The LLM couldn't recover from the noise.

Experiment 6: Canonical Backstory Approach

Setup: Pre-generate canonical backstory per character → Compare input against canonical using LLM
Result: 90% accuracy (9/10 correct)
Finding: This was our best approach. By comparing structured summaries rather than raw chunks, we eliminated most noise. The only error was a borderline case where the model was overly conservative.

Experiment 7: Canonical with Local LLM (Ollama)

Setup: Same as Experiment 6, but using llama3.2 (3B parameters) instead of Gemini
Result: 50% accuracy
Finding: The smaller local model wasn't capable enough for the nuanced comparison task. It defaulted to "consistent" for almost everything.

---

3. Handling Long Context (100k+ Words)

Novels in the dataset range from 140k to 460k words. We can't fit them in any LLM's context window, so we needed a chunking and retrieval strategy.

Chunking Decisions

We experimented with three chunk sizes:

- **500 words**: Too small. Important context got split across chunks. For example, if a character's backstory spans two paragraphs, we'd often get only half the relevant information.

- **2000 words**: Too large. Chunks became imprecise—they contained too much irrelevant text, diluting the signal.

- **1000 words with 200-word overlap**: This was our sweet spot. Chunks were large enough to preserve context but small enough to be specific.

Why Overlap Matters

We initially tried chunks without overlap and found that critical passages often fell at the boundary between chunks. The 200-word overlap ensures both chunks contain the complete context.

Canonical Backstory Generation

For the canonical approach, we don't rely on embedding-based retrieval. Instead, we find all chunks that mention a character by simple string matching (including handling aliases like "Tom Ayrton/Ben Joyce"). We then send all these chunks to the LLM and ask it to summarize:

> "Based ONLY on these excerpts, write a summary of what the novel tells us about [character]'s origins, key life events, relationships, and background."

This produces a clean, focused backstory of 2-4 paragraphs per character.

---

4. Distinguishing Causal Signals from Noise

The Core Problem

Looking at our error analysis across all experiments:

- **Inconsistent cases (actual=0)**: We consistently achieved 80-100% accuracy. It was probably because it cannot figure out whether the story was factually correct and just defaulted to inconsistency.

- **Consistent cases (actual=1)**: We struggled badly, starting at 0% and eventually reaching 62.5% with standard RAG. The model was finding contradictions where none existed.

What Helped

1. **Cross-encoder reranking**: The reranker learned to distinguish relevant evidence from superficial matches.
2. **Conservative prompting**: We explicitly instructed the model:
    - "Not mentioned" does NOT mean contradiction
    - "Seems unlikely" does NOT mean contradiction
    - Only mark inconsistent if there's EXPLICIT proof
3. **Canonical comparison**: By comparing two structured backstories instead of a backstory against raw chunks, we eliminated most noise.

What Didn't Work

- **Claim extraction**: We tried splitting backstories into individual claims and verifying each but it increased false positives.

- **NLI models**: Everything looked like a contradiction.

- **More retrieval**: Getting 20 chunks instead of 10 just added noise.

5. Limitations

Limitation 1: Character Aliases and Pronouns

Our character-based retrieval uses simple string matching. If the novel refers to a character by nickname, title, or pronoun without using their name, we might miss relevant passages.

Limitation 2: Rate Limits

We hit Gemini's free tier limit (5 RPM) frequently during experiments. This forced us to:

- Run most experiments on only 10 rows
- Make the pipeline resumable (saves progress after each row)
- Add exponential backoff with retries

Limitation 3: No Cross-Chapter Reasoning

Our approach is based on character-specific passages, not the full narrative arc. A backstory might be globally inconsistent (contradicting character development across the novel) while being locally plausible in the passages we retrieve.

Limitation 4: LLM Dependency

Both our canonical backstory generation and our classification steps require LLM calls. We tried using local models (Ollama) but accuracy dropped significantly. The task requires reasoning capabilities that smaller models lack.